Machine-Level Programming: Compound data types

Arquitectura de Computadores
Departamento de Engenharia Informática
Instituto Superior de Engenharia do Porto

Luís Nogueira (lmn@isep.ipp.pt)

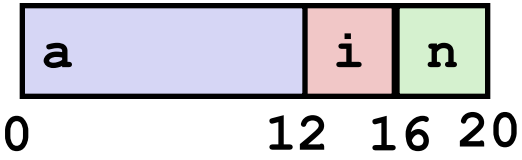
Today

- **Structures**
  - Allocation
  - Access
- **Data alignment**
- **Unions**

Structure allocation

```
struct rec {
  int a[3];
  int i;
  struct rec *n;
};
```

Memory layout



- **Concept**
  - Contiguously-allocated region of memory
  - Refer to members within structure by names
  - Members may be of different types
  - Compiler determines overall size and positions of fields

- **Fields ordered according to declaration**
  - Even if another ordering could yield a more compact representation

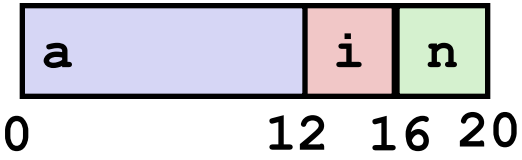
Declaring *struct* variables in C

- `struct rec r1, r2, r3;`
  - Declares and sets aside storage for three variables – r1, r2, and r3 – each of type *struct* rec
- `struct rec *ptr;`
  - Declares a pointer to an object of type *struct* rec
- `struct rec vec[25];`
  - Declares a 25-element array of *struct* rec
  - Allocates 25 units of storage, each one big enough to hold the data of one *struct* rec

Structure access in C

```
struct rec {
  int a[3];
  int i;
  struct rec *n;
};
```

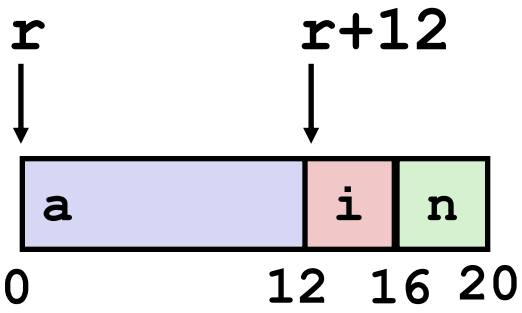
Memory layout



- **Given an instance of a structure:** `struct rec r1;`
  - `r1.i = val;`
- **Given a *pointer* to a structure:** `struct rec* r = &r1;`
  - Using * and . operators: `(*r).i = val;`
  - Or, use -> operator for short: `r->i = val;`

Structure access in Assembly

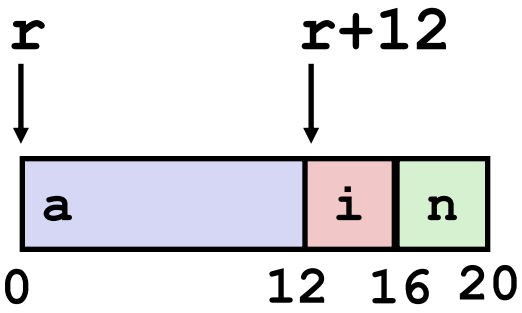
```
struct rec {
  int a[3];
  int i;
  struct rec *n;
};
```



- **Machine-level program has no understanding of structures in source C code**
- **Address indicates first byte of structure**
- **Access elements with offsets to the address of the structure**

Structure access in Assembly

```
struct rec {
  int a[3];
  int i;
  struct rec *n;
};
```



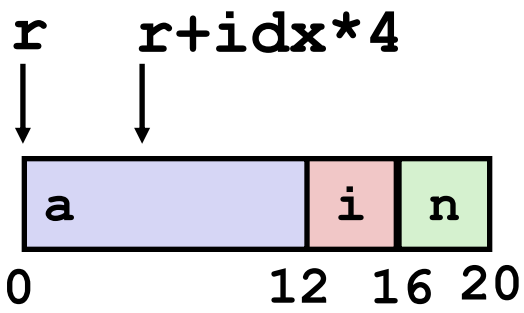

```
void set_i(struct rec *r, int val){
  r->i = val;
}
```

- **Access field with corresponding offset**

```
movl 8(%ebp), %eax          # r
movl 12(%ebp), %edx         # val
movl %edx, 12(%eax)         # Mem[r+12] = val
```

Generating pointer to structure member

```
struct rec {
  int a[3];
  int i;
  struct rec *n;
};
```




```
int *get_ap(struct rec *r, int idx)
{
  return &r->a[idx];
}
```

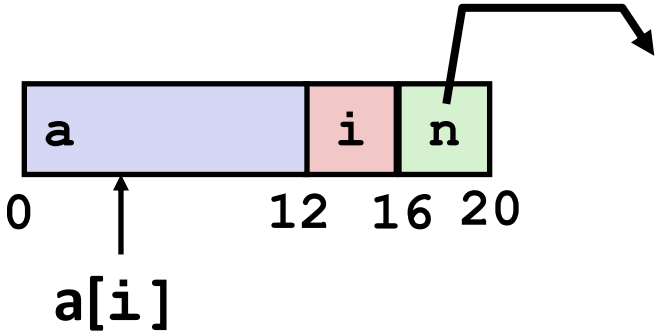
- **Add the field's offset to the structure's address**

```
movl   12(%ebp), %eax        # idx
sall   $2, %eax              # idx*4
addl   8(%ebp), %eax         # r+idx*4
```

Following linked list

```
struct rec {
  int a[3];
  int i;
  struct rec *n;
};
```

```
void set_val(struct rec *r, int val){
  while(r != NULL) {
    int i = r->i;
    r->a[i] = val;
    r = r->n;
  }
}
```




```
   movl   8(%ebp), %edx              # Get r
   movl   12(%ebp), %ecx             # Get val
.L17:                                # loop:
   movl   12(%edx), %eax             #   Get r->i
   movl   %ecx, (%edx,%eax,4)        #   r->a[i] = val
   movl   16(%edx), %edx             #   r = r->n
   testl  %edx, %edx                 #   Test r
   jne    .L17                       #   If != 0 goto loop
```

Today

- Structures
  - Allocation
  - Access
- **Data alignment**
- Unions

Motivation

- **Many computer systems place restrictions on the allowable addresses for the primitive data types**
  - Require that the address for some type of object must be a multiple of some value K (typically 2, 4, or 8)
- **The IA32 hardware will work correctly regardless of the alignment of data**
- **However, Intel recommends that data be aligned**
  - Simplifies the design of the hardware forming the interface between the processor and the memory system
  - Improves memory system performance

Alignment principles

- **Aligned data**
  - Primitive data type requires K bytes
  - Address must be multiple of K
- **Align enforcement by compiler**
  - Places *.align* directives in the assembly code indicating the desired alignment for global data
  - Library routines that allocate memory, such as *malloc*, must be designed so that they return a pointer that satisfies the worst-case alignment restriction (typically 4 or 8)
  - For code involving structures, **inserts gaps in structure** to ensure correct alignment of fields
- **Treated differently by IA32 Linux, x86-64 Linux, Windows and Mac OS!**

Specific cases of alignment (IA32)

- **1 byte: `char`**
  - No restrictions on address
- **2 bytes: `short`**
  - Lowest bit of address must be 0_2 (address must be multiple of 2)
- **4 bytes: `int`, `long`, `float`, `char *`, …**
  - Lowest 2 bits of address must be 00_2 (address must be multiple of 4)

Specific cases of alignment (IA32) (cont.)

- **8 bytes: `long long int`, `double`, …**
  - Windows (and most other OS's and instruction sets):
    - Lowest 3 bits of address must be 000_2 (multiple of 8)
  - Linux:
    - Lowest 2 bits of address must be 00_2 (multiple of 4)
    - i.e., treated the same as a 4-byte primitive data type

- **12 bytes: `long double`**
  - Windows, Linux: Lowest 2 bits of address must be 00_2 (multiple of 4)
    - i.e., treated the same as a 4-byte primitive data type

Specific cases of alignment (x86-64)

- **1 byte: `char`**
  - No restrictions on address
- **2 bytes: `short`**
  - Lowest 1 bit of address must be 0_2
- **4 bytes: `int`, `float`, …**
  - Lowest 2 bits of address must be 00_2
- **8 bytes: `long`, `double`, `char *`, …**
  - Lowest 3 bits of address must be 000_2
- **16 bytes: `long double`**
  - Linux: Lowest 3 bits of address must be 000_2
    - i.e., treated the same as a 8-byte primitive data type

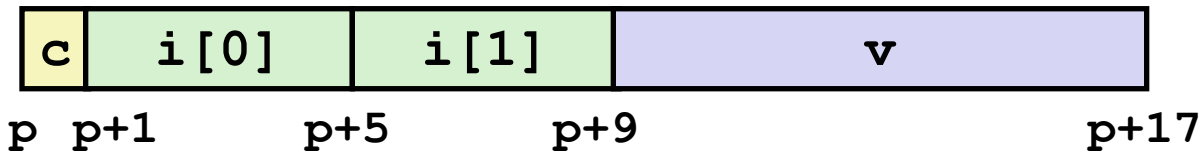
Data alignment in structures

- **Placement of structure in memory**
  - Each structure has an alignment requirement of K, where K is the size of the largest member of the structure
  - The starting address of the structure must be a multiple of K
  - The total size of the structure must be a multiple of K

- **Inside the structure**
  - The alignment requirement of each field type must be satisfied

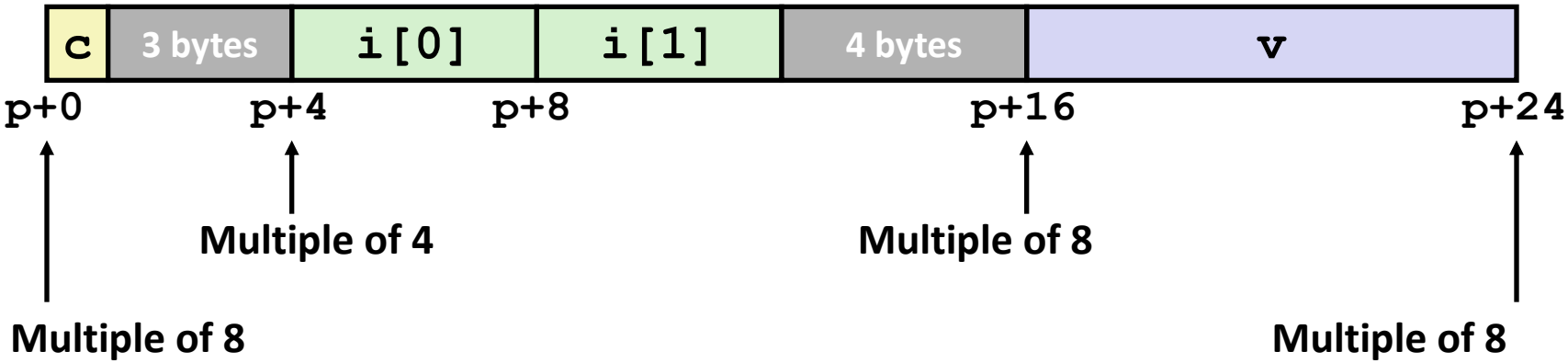
Placement of structure in memory

- **Unaligned data**




```
struct S1 {
  char c;
  int i[2];
  long long v;
} *p;
```

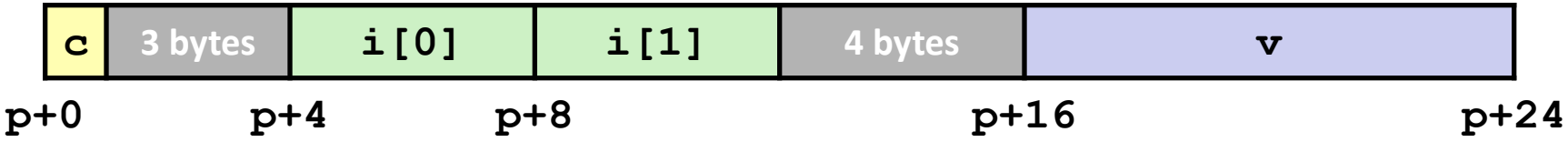
- **Aligned data under Windows or x86-64**
  - K = 8, due to *long long* element



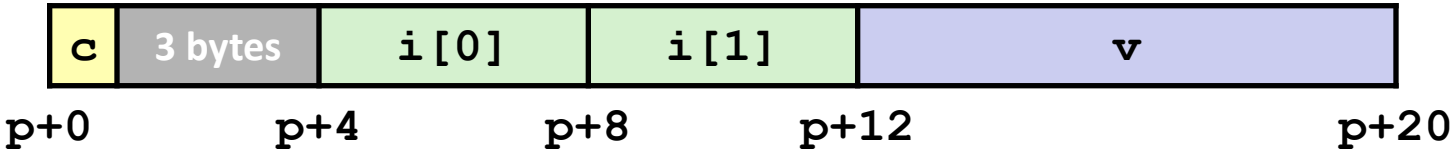
Alignment requirements of members

```
struct S1 {
  char c;
  int i[2];
  long long v;
} *p;
```

- **For each member**
  - Satisfy alignment requirement of K
- **x86-64 or IA32 Windows**
  - K = 8, due to *long long* element



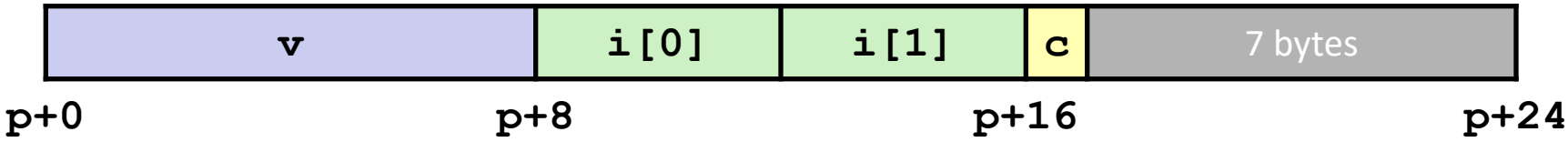
- **IA32 Linux**
  - K = 4; *long long* treated like a 4-byte data type



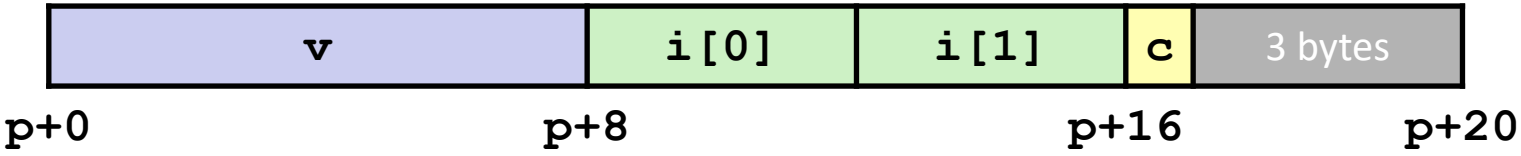
Overall alignment requirement

```
struct S2 {
  long long v;
  int i[2];
  char c;
} *p;
```

- **Overall structure size**
  - Must be multiple of K
  - K = largest alignment requirement
- **x86-64 or IA32 Windows**
  - K = 8, due to *long long* element



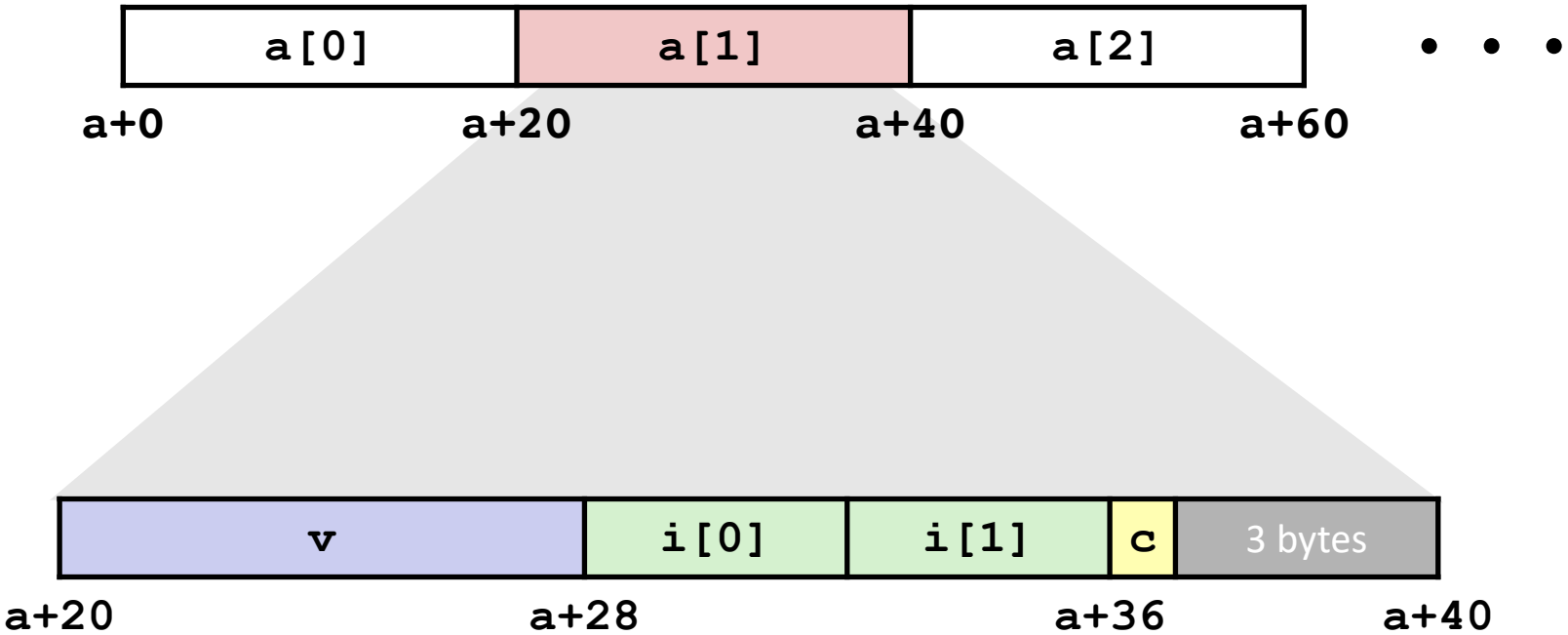
- **IA32 Linux**
  - K = 4; *long long* treated like a 4-byte data type



Arrays of structures

```
struct S2 {
  long long v;
  int i[2];
  char c;
} a[10];
```

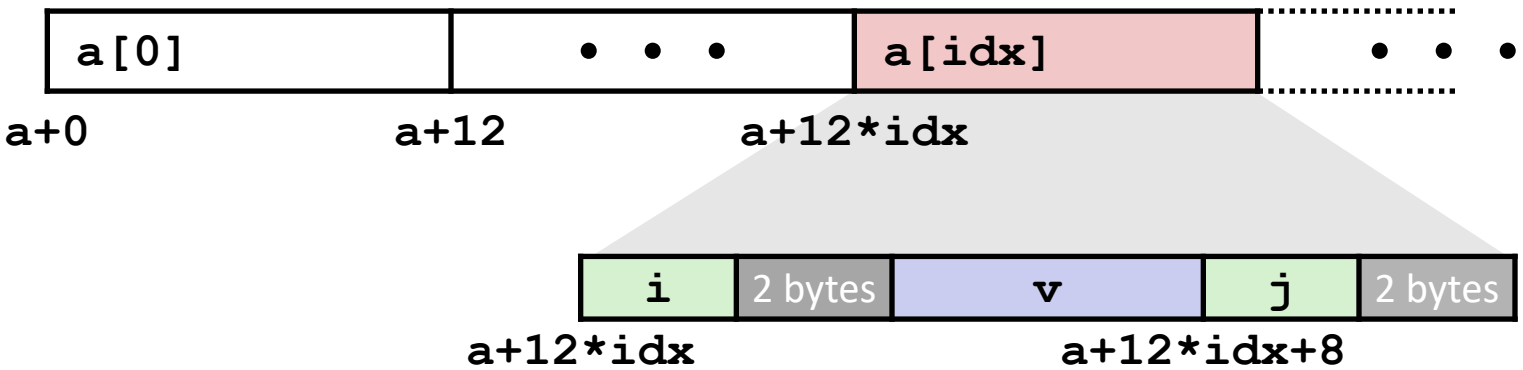
- **Contiguously allocated region of *n* * `sizeof` (*struct x*) bytes**
  - Overall structure length multiple of K
  - Satisfy alignment requirement for every element



Accessing array elements

```
struct S3 {
  short i;
  int v;
  short j;
} a[10];
```

- **Compute array offset 12 * idx**
  - `sizeof(S3) = 12`, including alignment spacers
- **Element `j` is at offset 8 within structure**




```
short get_j(int idx){
  return a[idx].j;
}
```

```
movl 8(%ebp), %eax              # idx
leal (%eax,%eax,2),%eax         # 3*idx
movswl a+8(,%eax,4),%eax
```

Practice problem

- **For each of the following structure declarations determine, under Linux/IA32:**
  - the offset of each field;
  - the total size of the structure;
  - and its alignment requirement.

A. `struct P1 {int i; char c; int j; char d;};`

B. `struct P2 {int i; char c; char d; int j;};`

C. `struct P3 {short w[3]; char c[3];};`

D. `struct P4 {short w[3]; char *c[3]};`

E. `struct P5 {struct P1 a[2]; struct P2 *p;};`

Practice problem

A. `struct P1 {int i; char c; int j; char d;};`

Field	i	c	[gap]	j	d	[gap]	Total	Alignment
Offset	0	4	5	8	12	13	16	K = 4

B. `struct P2 {int i; char c; char d; int j;};`

Field	i	c	d	[gap]	j	Total	Alignment
Offset	0	4	5	6	8	12	K = 4

C. `struct P3 {short w[3]; char c[3];};`

Field	w	c	[gap]	Total	Alignment
Offset	0	6	9	10	K = 2

Practice problem

D. `struct P4 {short w[3]; char *c[3];};`

Field	w	[gap]	c	Total	Alignment
Offset	0	6	8	20	K = 4

E. `struct P3 {struct P1 a[2]; struct P2 *p;};`

Field	a	p	Total	Alignment
Offset	0	32	36	K = 4

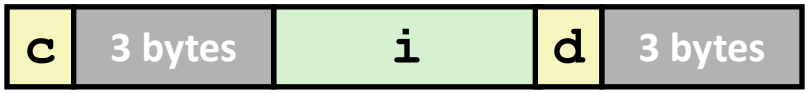
Saving space

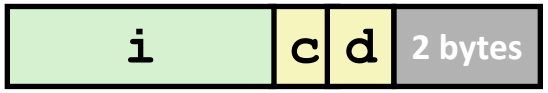
- **Put large data types first**

```
struct S6 {
  char c;
  int i;
  char c;
} *p;
```

```
struct S7 {
  int i;
  char c;
  char d;
} *p;
```

- **Effect on IA32 Linux (K=4)**





12 bytes **8 bytes**

- **This strategy *can* save space in certain structures**

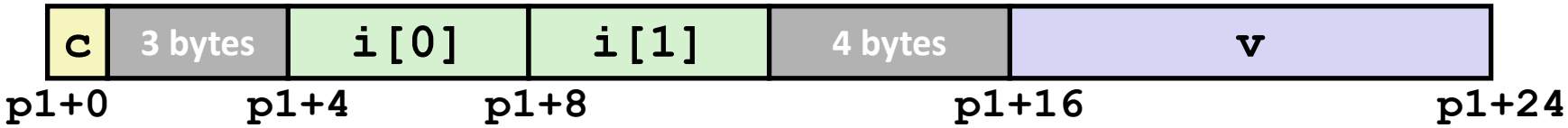
Saving space

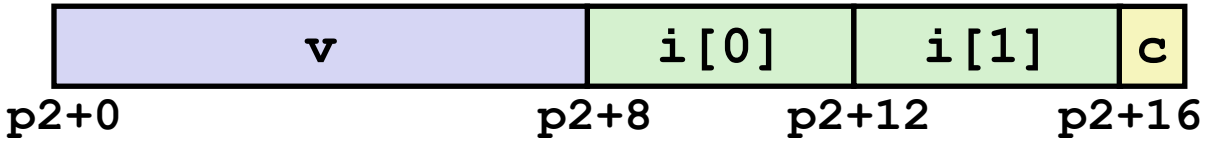
- **Put large data types first**

```
struct S1 {
  char c;
  int i[2];
  long long v;
} *p1;
```

```
struct S2 {
  long long v;
  int i[2];
  char c;
} *p2;
```

- **Effect on x86-64 (K=8)**





Unfortunately, doesn't satisfy requirement that struct's *total* size is a multiple of K

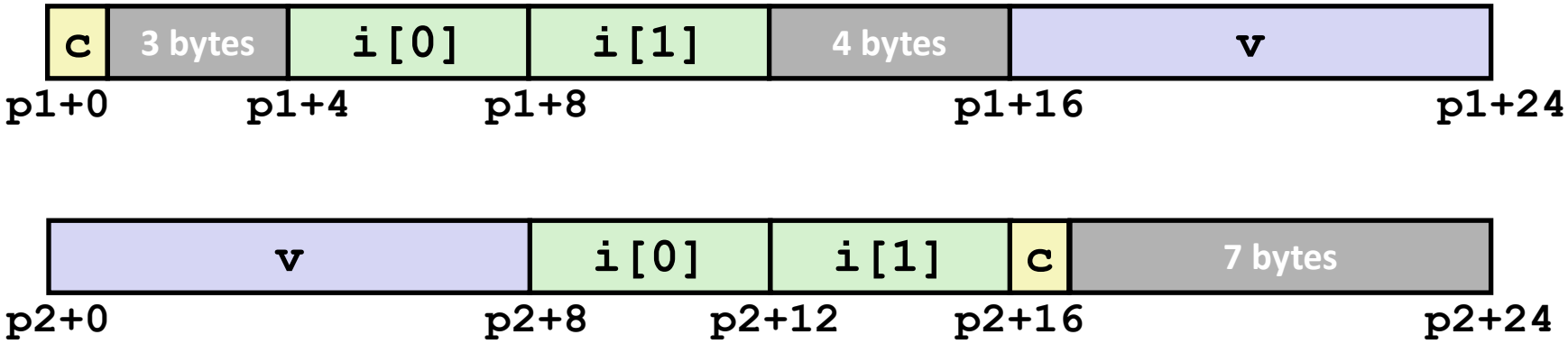
Saving space

- **Put large data types first**

```
struct S1 {
  char c;
  int i[2];
  long long v;
} *p1;
```

```
struct S2 {
  long long v;
  int i[2];
  char c;
} *p2;
```

- **Effect on x86-64 (K=8)**



Today

- Structures
  - Allocation
  - Access
- Data alignment
- **Unions**

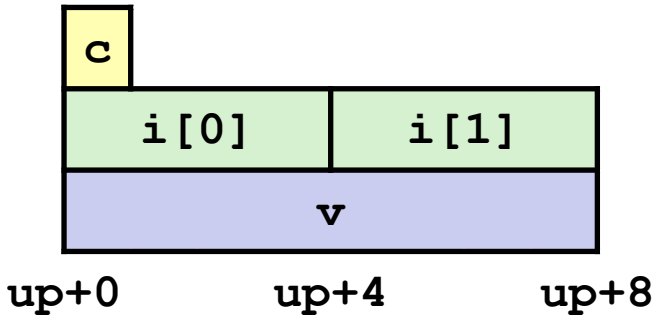
Unions

- **Provide a way to circumvent the type system of C**
  - Allow a single object to be referenced according to multiple types
- **The syntax of a union declaration is identical to that for structures, but its semantics are very different**
  - Rather than having the different fields reference different blocks of memory, they all reference the same block
- **The overall size of a union equals the maximum size of any of its fields**
  - Important when memory is more scarce, such as in embedded systems

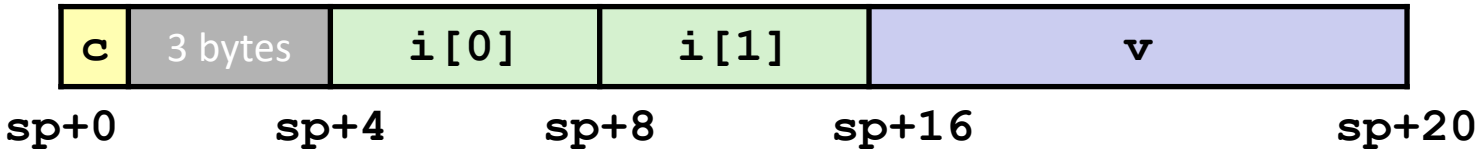
Union allocation

- **Allocated according to largest element**
- **Can only use one member at a time**

```
union U1 {
  char c;
  int i[2];
  long long v;
} *up;
```




```
struct S1 {
  char c;
  int i[2];
  long long v;
} *sp;
```



Accessing union members

```
union Data{
  int i;
  float f;
  char str[20];
} data;

data.i = 10;
printf("data.i   : %d\n", data.i);      /* 10 */

data.f = 220.5;
printf("data.f   : %f\n", data.f);      /* 220.5 */

strcpy(data.str,"C Programming");
printf("data.str : %s\n", data.str);    /* C Programming */
```

Accessing union members

```
union Data{
  int i;
  float f;
  char str[20];
} data;

data.i = 10;
data.f = 220.5;
strcpy(data.str,"C Programming");

printf("data.i   : %d\n", data.i);      /* 1917853763 */

printf("data.f   : %f\n", data.f);      /* 4122360580327.00 */

printf("data.str : %s\n", data.str);    /* C Programming */
```

Accessing union members

- Unions can be useful in several contexts but they can also lead to nasty bugs
- No checking is done to make sure that the right sort of use is made of the members
- It's up to the programmer to keep track of whatever type is put into it

Common use

- **Combining them with *structs***
  - Produce variable types based on a common core, but then varying in parts where appropriate

```
typedef union {
  int units;
  float kgs;
} amount;
```

```
typedef struct {
   char selling[15];
   float unitprice;
   int unittype;
   amount howmuch;
 } product;
```

- **Still require a lot of discipline to ensure that only the active member is accessed at any time**

Summary

- **Structures**
  - Elements packed into single region of memory
  - Access using offsets determined by compiler
  - Possible require internal and external padding to ensure alignment
- **Combinations**
  - Can nest structure and array code arbitrarily
- **Unions**
  - Overlay declarations
  - Way to circumvent type system